

## 单表查询

SQL 查询语句仅一种 SELECT 句型

SELECT ... <sup>所有行</sup> ALL / <sup>消除查询结果中重复行</sup> DISTINCT 字段名 AS 别名  
FROM 表名 AS 表别名  
WHERE ...  
GROUP BY 用于分组的列名 HAVING ...  
ORDER BY 用于排序的列名 ASC / DESC  
LIMIT ...

- “字段名” AS “别名” 可让别名代替字段名显示在查询结果中
- “表名” AS “表别名” 可让表别名代替表名显示在查询结果中
- WHERE 选取符合检索条件的行, HAVING 用于筛选分组查询后满足条件的行
- ORDER BY 对查询结果排序, ASC 表示升序而 DESC 表示降序 (默认为升序)
- LIMIT 限制查询结果的行数

### 无条件查询

查询指定列 SELECT 列1, 列2, ... FROM 表 ;

这里列的显示顺序可依需要选择, 可与基本表中的不一致

查询所有列 SELECT \* FROM 表 ;

\* 选中所有列, 但此时列的显示顺序遵循原表顺序, 若要指定显示顺序就要按实际需要依次输入所有列名

### 查询经计算的值

查询学生的学号和年龄

SELECT sno, YEAR(SYSDATE()) - YEAR(birthday) FROM student;

### 列的别名

SELECT sno AS 学号, YEAR(SYSDATE()) - YEAR(birthday) AS 年龄 FROM 年龄;

另举一例, SELECT sno AS 学号, sname AS 姓名, sex AS '性别(男,女)' FROM student;

若别名中有空格、星号、关键字等特殊字符, 原样输出时要用引号

SELECT "3+2=", 3+2;

### 消除重复行

SELECT DISTINCT 不希望出现重复内容的列的列名 FROM 表 ;  
(即删除重复行)

## 条件查询

### 比较运算

查询总学分<sup>不等于</sup>大于50的学生姓名和总学分

```
SELECT sname, totalcredit FROM student WHERE totalcredit <> 50;
```

### 范围运算

查询学分在40与49之间<sup>不包括40和49</sup>的学生学号和总学分。

```
SELECT sno, totalcredit FROM student WHERE totalcredit NOT BETWEEN 40 AND 49;
```

### 集合运算

查询考试成绩为80、85或90的学生学号

```
SELECT sno FROM score WHERE grade IN (80, 85, 90);
```

查询不是计算机系、物理系、数学系的学生信息

```
SELECT * FROM student WHERE dept NOT IN ('计算机', '物理', '数学');
```

模糊查询 (%: 任意长(含长度为0)字符串, -: 任意单个字符)

查询姓名以"林"开头的学生信息

```
SELECT * FROM student WHERE sname LIKE '林%';
```

查询姓名中有"林"字的学生信息

```
SELECT * FROM student WHERE sname LIKE '%林%';
```

查询姓名长度至少是两个字符且倒数第二个字符必是"林"的学生信息

```
SELECT * FROM student WHERE sname LIKE '%林-';
```

若待查内容本身含有"-", 则应使用转义字符'\ 将其转为普通字符

在course表中插入两条记录, 课程名分别为"DBDESIGN"和"DB-DESIGN"

```
INSERT INTO course(cno, cname) VALUES ('401', 'DBDESIGN'), ('402', 'DB-DESIGN');
```

查询以"DB-"开头的课程的情况

```
SELECT * FROM course WHERE cname LIKE 'DB\_%';
```

\为默认, 若要使用其他的例如@

```
SELECT * FROM course WHERE cname LIKE 'DB@_%' ESCAPE '@';
```

## 空值运算

这里 NULL 指不存在/不确定/不可用的数据, 未被赋值的字段、长度为0的字符串都是空值

赋空值:

① 赋两个连续的单引号

② 赋 NULL

查询 course 表中学分为空的记录

```
SELECT * FROM course WHERE credit IS NULL;
```

空值不能直接参与运算, 让其改变需使用 IFNULL (expr1, expr2) 函数  $\Leftrightarrow$   $\begin{cases} \text{expr1 非空就返回 expr1} \\ \text{expr1 为空就返回 expr2} \end{cases}$

给所有学生的总学分加 10

```
UPDATE course
```

```
SET credit = IFNULL(credit, 0) + 10
```

```
WHERE cname LIKE 'DB%';
```

另外, 在定义表时可以用 default 定义一个默认值来代替空值

## 混合运算

查询计算机系或通信工程系总学分大于 50 的学生姓名、系别、总学分

```
SELECT sname, dept, totalcredit
```

```
FROM student
```

```
WHERE dept IN ('计算机', '通信工程') AND totalcredit > 50;
```

查询选修课程 101 或 102, 成绩在 85 和 95 之间学生的学号、课程号、成绩

```
SELECT sno, cno, grade
```

```
FROM score
```

```
WHERE cno IN ('101', '102') AND grade BETWEEN 85 AND 95;
```

## 组函数 (COUNT, SUM, AVG, MAX, MIN)

COUNT() 统计满足条件的记录的个数或所选列的个数

使用 DISTINCT 就表示计算时要消除指定列中的重复值, 否则表示允许重复

COUNT(\*) 对表中行的数目进行计数, 不论是否包含空值 NULL

COUNT(列) 对特定列中的行进行计数, 忽略空值 NULL

查询学生总人数

```
SELECT COUNT(*) AS 总人数 FROM student;
```

查询选修了课程的学生人数

```
SELECT COUNT(DISTINCT sno) AS 选修人数 FROM course;
```

查询计算机系学生的平均学分

```
SELECT AVG(totalcredit) FROM student WHERE dept = '计算机';
```

查询选修了课程号为 101 课程的学生们的最高、最低、平均成绩

```
SELECT MAX(grade), MIN(grade), AVG(grade) FROM score WHERE cno = '101';
```

分组查询 (GROUP BY 将查询结果表中的各行按列分组, 将列值相等的归于一组)

在有 GROUP BY 的查询语句中, SELECT 后所有字段列表 (组函数除外) 均应包含在 GROUP BY 句中。

选项与分组应具有一致性

若后有 HAVING, 则代表满足 HAVING 指定条件的组才能输出 (HAVING 作用于分组后的组)

|                                                                                                                |                                                                                                                |
|----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <p>分组</p> <p>查询各门课程的平均成绩与总成绩</p> <pre>SELECT cno, AVG(grade), SUM(grade) FROM score GROUP BY cno;</pre>        |                                                                                                                |
| <p>查询各系的人数</p> <pre>SELECT dept, COUNT(*) FROM student GROUP BY dept;</pre>                                    | <p>查询各系的男女生人数</p> <pre>SELECT dept, sex, COUNT(*) FROM student GROUP BY dept, sex;</pre>                       |
| <p>查询人数在 10 人以上的系别及人数</p> <pre>SELECT dept, COUNT(*) FROM student GROUP BY dept HAVING COUNT(*) &gt; 10;</pre> | <p>查询计算机系的男女生人数</p> <pre>SELECT dept, sex, COUNT(*) FROM student WHERE dept = '计算机系' GROUP BY dept, sex;</pre> |

WHERE 作用于基本表或视图, 选择满足条件的行

HAVING 作用于组, 选择满足条件的组

## 排序查询

若不指定查询结果的显示顺序, DBMS 就按行在表中的先后次序输出查询结果

ASC 升序排序(默认)时, 含有空值的行排在最前

'101' 或 '102'

查询选修课程<sup>V</sup>的学生学号、课程号、成绩, 结果按学号升序、成绩降序排序

```
SELECT sno, cno, grade
```

```
FROM score
```

```
WHERE cno = '101' OR cno = '102'
```

```
ORDER BY sno ASC, grade DESC;
```

## 限制查询结果数量

### ① 限制返回结果的记录数

LIMIT 表示限定获取的最大记录数量, 没有OFFSET就从第一条记录开始获取

有OFFSET(偏移量)就表示设置了从哪条记录开始获取(第1条记录偏移量是0)

查询课程号为101的课程成绩排前3名的学生学号、课程号、成绩

```
SELECT sno, cno, grade
```

```
FROM score
```

```
WHERE cno = '101'
```

```
ORDER BY grade DESC
```

```
LIMIT 3;
```

查询课程号为101的课程成绩排6~10名的学生学号、课程号、成绩

```
SELECT sno, cno, grade
```

```
FROM score
```

```
WHERE cno = '101'
```

```
ORDER BY grade DESC
```

```
LIMIT 5, 5;
```

偏移量

## ② 分页显示返回的记录

每页显示 `pageSize` 条记录, 显示第 `pageNo` 页:

`LIMIT (pageNo - 1) * pageSize, pageSize;`  
从哪条记录开始显示      每页显示行数

每页显示 10 条记录, 分页显示 `student` 表所有记录:

第 1 页 → `LIMIT 0, 10;`

第 2 页 → `LIMIT 10, 10;`

第 3 页 → `LIMIT 20, 10;`

## ③ 排序后限量更新或删除数据

把课程号 101 的成绩按学号升序排, 更新前 3 条成绩 + 10 分

```
UPDATE score
SET grade = grade + 10
WHERE cno = '101'
ORDER BY sno ASC
LIMIT 3;
```

删除 `course` 表中 `ctime` 为 `NULL` 的两条记录

```
DELETE FROM course
ORDER BY ctime ASC
LIMIT 2;
```

## 综合练习

数据库 mytest, 表 zlgc (pid, pname, school, category, head, grade)

(1) 统计每个学校获批项目数, 并按降序排列, 哪个学校获批最多?

```
SELECT school, COUNT(*)  
FROM zlgc  
GROUP BY school  
ORDER BY COUNT(*) DESC  
LIMIT 1;
```

(2) 统计每个学校在不同类别项目上分别获批多少个

```
SELECT school, category, COUNT(*)  
FROM zlgc  
GROUP BY school, category;
```

(3) “课程思政示范课程”项目哪个学校获批最多?

```
SELECT school, COUNT(*)  
FROM zlgc  
WHERE pnamecategory = '课程思政示范课程'  
GROUP BY school  
ORDER BY COUNT(*) DESC  
LIMIT 1;
```

(4) 获批“课程思政示范课程”50项以上的学校有哪些?

```
SELECT school, COUNT(*)  
FROM zlgc  
WHERE pnamecategory = '课程思政示范课程'  
GROUP BY school  
HAVING COUNT(*) > 50;
```